

Conceito de Sistema Distribuído

Um sistema distribuído é um conjunto de elementos que se comunicam através de uma rede de interconexão e que utilizam software de sistema distribuído. Cada elemento é composto por um ou mais processadores e uma ou mais memórias.

Características que complicam o projeto de sistemas distribuídos:

- ausência de memória global
- ausência de clock global
- imprevisibilidade no retardo de mensagens

Características principais:

- Comunicação via rede de interconexão.
- Coordenação e integração através de software específico.
- Transparência para o usuário: não há percepção de onde está o recurso ou como ele é processado.

Computador Paralelo

Um computador paralelo é uma coleção de processadores que cooperam de forma coordenada para resolver problemas mais rapidamente. Ele pode ser entendido como um caso particular de sistema distribuído, mas com foco em desempenho e aceleração de tarefas.

Diferença chave:

- Sistemas distribuídos: têm como objetivo integrar recursos, prover disponibilidade, escalabilidade e tolerância a falhas.
- Sistemas paralelos: buscam essencialmente velocidade no processamento.

Por isso, podemos dizer que sistemas paralelos são um subtipo de sistemas distribuídos, diferenciando-se pelo objetivo principal.

Arquiteturas Paralelas

- **Definição:** Arquiteturas compostas por mais de um processador trabalhando de forma cooperativa.
- **Motivação:** Mesmo com os avanços tecnológicos, há problemas que **um processador isolado não consegue resolver em tempo hábil.**

Exemplos: modelagem climática, análise genômica, simulações físicas complexas.

Benefícios das Arquiteturas Paralelas

- **Desempenho:** ganho de velocidade pela divisão de tarefas.
- **Disponibilidade:** maior confiabilidade, pois falhas em um nó podem ser compensadas por outros.
- **Custo/Desempenho:** melhor relação de investimento em comparação a superprocessadores únicos.
- **Crescimento incremental:** possibilidade de adicionar mais processadores ou máquinas conforme necessárias.
- **Uso de ciclos ociosos:** aproveitamento de recursos que estariam inativos.

High Performance Computing (HPC)

- **Definição:** modelo de computação de alto desempenho, voltado para processar **um grande volume de operações numéricas intensivas**.
- **Métrica:** desempenho medido em **número de operações de ponto flutuante por segundo (Flops/s)**.
- **Características:**
 - Uso **dedicado:** os recursos computacionais ficam alocados especificamente para a tarefa de alto desempenho.
 - Normalmente utilizados em **supercomputadores, clusters ou MPP (massively parallel processors)**.
 - Aplicações típicas: simulações climáticas, genômica, modelagem molecular, radioastronomia etc.
- **Exemplo prático:** Supercomputador *El Capitan* (EUA), capaz de mais de 2,7 exaflops.

High Throughput Computing (HTC)

- **Definição:** modelo de computação de alto rendimento, voltado para maximizar a execução de **grande quantidade de tarefas independentes**.
- **Métrica:** desempenho medido em **quantidade de programas concluídos por hora (programas/hora)**.
- **Características:**
 - Utiliza **ciclos ociosos de máquinas comuns** (desktops, servidores, clusters).
 - Não busca acelerar uma única tarefa complexa, mas **aumentar a taxa de tarefas concluídas** em um intervalo de tempo.

- É adequado para workloads massivamente paralelos, como análises de dados independentes.
- **Exemplo prático:** grades computacionais (*grids*) que aproveitam o tempo ocioso de estações de trabalho em universidades ou empresas.

Comparação HPC x HTC

Aspecto	HPC (High Performance)	HTC (High Throughput)
Unidade de medida	Flops/s (operações numéricas)	Programas/hora (tarefas concluídas)
Uso típico	Supercomputadores, clusters dedicados	Grades computacionais, clusters aproveitando ciclos ociosos
Objetivo principal	Resolver um problema complexo mais rápido	Resolver muitos problemas menores em maior quantidade
Perfil de tarefas	Computações científicas pesadas	Execuções repetitivas ou independentes

Módulo 2 - Escopo e Decisões de Projeto em Sistemas Distribuídos

Transparência: a ilusão de um sistema único

O conceito de **transparência** é central em sistemas distribuídos: o usuário e o programador devem perceber o sistema como se fosse **um único computador**, mesmo que haja múltiplas máquinas conectadas. Ela pode ocorrer em diferentes níveis e dimensões:

Tipos de Transparência (ISO RM-ODP)

Transparência de Acesso

- **Definição:** recursos locais e remotos podem ser acessados de forma **idêntica**, com as mesmas operações.
- Permite que recursos locais e remotos sejam acessados através de operações idênticas.
- **Exemplo:** NFS permite copiar arquivos locais e remotos com o mesmo comando; FTP não, pois exige comandos específicos.
- **Interligação:** base para todas as outras transparências, pois sem uniformidade de acesso não há ilusão de sistema único.

Transparência de Localização

- **Definição:** o usuário não deve saber **onde** o recurso está.
- Os sistemas transparentes quanto à localização geralmente possuem um servidor de nomes, que fará o mapeamento entre o nome e o endereço.
- **Exemplo:** servidor de nomes faz o mapeamento entre nome lógico e endereço físico.
- **Relação:** complementa a transparência de acesso
- não importa **como** acessar (acesso) nem **onde** está (localização).

Transparência de Concorrência

- **Definição:** múltiplos usuários podem acessar o mesmo recurso sem perceberem conflitos, preservando a **coerência**.
- Os usuários não devem notar que existem outros usuários no sistema. Se dois usuários acessam simultaneamente um mesmo recurso, o sistema deve garantir a coerência.
- Em sistemas distribuídos, devem ser garantidas as mesmas condições de concorrência de uma única máquina
- **Problema:** mesmo em máquinas únicas é difícil; em sistemas distribuídos, a ausência de clock global agrava.
- **Exemplo:** controle de acesso simultâneo a um banco de dados.
- **Relação:** depende de mecanismos de sincronização → conecta-se à transparência de replicação e falha.

Transparência de Replicação

- **Definição:** várias cópias de um recurso devem parecer uma só, mantendo a **consistência**.
- **Exemplo:** múltiplos servidores de arquivos com replicação automática.
- **Relação:** reforça a concorrência (vários acessos em cópias) e aumenta a tolerância a falhas.
- Permite que diversas cópias do mesmo recurso sejam utilizadas sem que os usuários ou programadores estejam conscientes de sua existência.
- O sistema garante, então, a coerência entre as diversas cópias.

Transparência de Falha

- **Definição:** mesmo diante de falhas, a tarefa deve ser completada (detecção, recuperação e redundância).
- **Exemplo:** reenvio automático de mensagens em RPC.
- **Relação:** depende da replicação (cópias substituem recursos falhos) e da migração (processos movem-se para nós ativos).
- Permite que as tarefas sejam completadas, mesmo na ocorrência de falhas.
- Um sistema deste tipo deve ter um mecanismo de detecção e recuperação de falhas

Transparência de Migração

- **Definição:** recursos (dados, processos, computação) podem mudar de lugar sem afetar o usuário.
- **Exemplo:** balanceamento de carga movendo processos para outro nó.
- **Relação:** engloba transparência de localização (pois envolve realocação) e conecta-se à de falha (migração após crash).
- Os recursos podem trocar de lugar à vontade no sistema. Um sistema transparente quanto à migração é necessariamente transparente quanto à localização, mas, além disso, deve observar outras características de projeto.
- Migração de dados
 - Transferência de arquivos: Quando um usuário necessita acessar um arquivo x, o arquivo x completo é transferido para a sua máquina local. Se houver alterações, o arquivo deve ser transferido de volta ao site origem
 - Transferência de partes do arquivo: Somente as partes do arquivo que serão acessadas são realmente transferidas.
- Migração de computação
 - Quando se necessita de um grande volume de dados que se encontra em outro nodo, é mais eficiente transferir a computação do que transferir os dados.
 - A migração de computação pode ser feita via RPC ou pelo envio de mensagem
- Migração de processos
 - A migração de um processo, depois de iniciada a sua execução, pode ser justificada pelas seguintes razões:
 - Balanceamento de carga
 - Crash de um nodo
 - Preferências de hardware
 - Preferências de software
- Quando um recurso migra, pode haver solicitações em andamento no sistema para este componente, que não tomaram ainda conhecimento da nova localização.
- Em um sistema com dependência residual, os nodos guardam um histórico do movimento dos recursos, para que o processo que possua sua localização antiga (nome antigo) possa localizá-lo.
- Em sistemas sem dependência residual, a requisição falha e o nodo origem da solicitação, sabendo que o sistema permite migração, solicita a nova localização ao sistema

Transparência de Paralelismo

- **Definição:** o sistema decide automaticamente quantos e quais recursos alocar para uma aplicação.

- **Exemplo:** execução distribuída variando número de processadores conforme carga.
- **Relação:** depende da escalabilidade e da concorrência, pois lida com múltiplas execuções em paralelo.
- O próprio sistema decide que recursos (e.g processadores) alocar a uma aplicação distribuída de maneira que critérios de otimização sejam atendidos (balanceamento de carga, tempo de resposta etc.).
- O usuário não deve interferir nesta escolha.
- Logo, dependendo da carga do sistema, o número de recursos alocados a uma aplicação pode variar de uma execução para outra.

Heterogeneidade

- Em um sistema distribuído, podemos ter os seguintes tipos de heterogeneidade:
 - Rede de interconexão
 - Hardware
 - Sistema operacional
 - Linguagem de programação
- Os diversos tipos de redes de interconexão que compõem o sistema geralmente são mascarados pelo uso do mesmo protocolo.
- Os diferentes hardwares de computadores são mascarados (de maneira restrita) quando é definida uma representação padrão de dados para troca de mensagens.
- Os diferentes sistemas operacionais são mascarados pelo uso de interfaces de comunicação de alto nível
- Middleware: é uma camada de software (software layer) que provê um modelo computacional uniforme, mascarando a heterogeneidade da plataforma de hardware e software.

Abertura (Openness)

- Representa o **grau de extensibilidade** do sistema.
- Alcançada pela **documentação e publicação das interfaces de software** dos componentes → facilita padronização.
- Interfaces geralmente divulgadas em **RFCs (Requests for Comments)**, que são a base de protocolos abertos da Internet.
- **Síntese:** quanto mais aberto, maior a interoperabilidade e a evolução do sistema.

Segurança

Envolve três dimensões fundamentais:

- **Confidencialidade:** impede acesso por usuários não autorizados.
- **Integridade:** protege contra alterações ou corrupção indevida da informação.

- **Disponibilidade:** garante que os recursos permaneçam acessíveis quando necessários.
- A segurança é transversal e interage diretamente com **acesso, concorrência e falhas**.
- A segurança em sistemas distribuídos envolve 3 componentes:
 - Confidencialidade: proteção contra indivíduos não autorizados
 - Integridade: proteção contra alterações e corrupção da informação
 - Disponibilidade: proteção contra a indisponibilidade da informação

Tratamento de Falhas

- A presença de múltiplos processadores aumenta a confiabilidade, mas na prática a falha de um componente pode comprometer todo o sistema.
- Técnicas usuais:
 - **Detecção** (identificar falhas).
 - **Mascaramento** (ocultar efeitos de falhas).
 - **Tolerância** (garantir funcionamento parcial).
 - **Recuperação** (restaurar após falha).
 - **Redundância** (manter cópias de recursos ou processos).
 - Conectado à **replicação e migração**, pois ambas são mecanismos para mitigar falhas.

Concorrência

- Sistemas distribuídos permitem múltiplos acessos simultâneos a recursos compartilhados.
- Necessário empregar **mecanismos de sincronização** (ex.: locks, semáforos, algoritmos de exclusão mútua).
- Relaciona-se diretamente à **transparência de concorrência** já vista: o usuário não deve perceber conflitos.

Escalabilidade

- **Definição intuitiva:** um sistema eficiente com 200 máquinas deve continuar eficiente com 200.000.
- **Níveis:** arquitetura, sistema operacional, linguagem de programação e aplicação.
- **Estratégias:**
 - Fixar o número de recursos por entidade gerenciadora.
 - Aumentar proporcionalmente o número de gerentes.
 - Usar **estruturas hierárquicas e algoritmos descentralizados** para evitar gargalos.
- Relaciona-se à **transparência de paralelismo**, pois o crescimento exige alocação dinâmica de recursos.

Desempenho

- A expectativa é de maior performance com mais máquinas.
- **Limitação principal: custo de comunicação**, que inclui o protocolo e a transmissão física.
- O problema principal é que, reduzindo a comunicação, podemos estar reduzindo o paralelismo e, conseqüentemente, reduzindo a performance.
- **Trade-off**: reduzir comunicação pode reduzir também o paralelismo → é preciso um **equilíbrio**.
- **Granulosidade: determina o elemento básico que será executado em paralelo.**
 - **fin**: as instruções ou um conjunto pequeno de instruções são executados em paralelo
 - **média**: as funções são executadas em paralelo (a nível de threads).
 - **grossa**: processos são executados em paralelo (grande quantidade de código para cada processo).
- Conecta-se à **paralelização e escalabilidade**, pois influencia a forma como processos são distribuídos.

Módulo 3 - Modelos Arquiteturais e de Sistema em Sistemas Distribuídos

Modelos Arquiteturais

- **Definição**: descrevem como os componentes de um sistema distribuído são dispostos na rede e como se relacionam.
- Baseiam-se em **processos ou objetos** como elementos fundamentais .

Tipos de Processos

- **Cliente**: solicita a execução de uma tarefa.
- **Servidor**: executa a tarefa solicitada.
- **Peer (par)**: coopera de maneira simétrica, podendo atuar tanto como cliente quanto como servidor .

A evolução tecnológica trouxe variações: clientes podem, por exemplo, descarregar código do servidor e executá-lo localmente (como *applets* em navegadores).

Níveis de Software

A organização típica de um sistema distribuído pode ser vista em **camadas**:

1. **Hardware**
2. **Sistema Operacional**

3. **Middleware** (responsável por mascarar a heterogeneidade e oferecer APIs uniformes).
4. **Aplicações e Serviços** .

Modelos de Arquitetura de Sistema

Segundo Coulouris, Dollimore e Kindberg, destacam-se cinco modelos principais

Cliente/Servidor

- Estrutura mais tradicional.
- Cliente **inicia as interações** → envia requisições.
- Servidor responde com o serviço solicitado.
- Exemplo: web browser solicitando página a um servidor web.

Processos Peer-to-Peer (P2P)

- Todos os processos têm papel **simétrico**: podem fornecer e consumir serviços.
- Elimina a hierarquia fixa entre cliente e servidor.
- Exemplo: compartilhamento de arquivos em redes P2P.

Agentes Móveis

- Programas que podem **migrar pela rede** para executar tarefas em outros nós.
- Vantagem: levar a computação até os dados, reduzindo custo de comunicação.

Network Computers

- Dispositivos com baixa capacidade de processamento local, dependentes de servidores para executar tarefas.
- Exemplo: *thin clients* que acessam aplicações via rede.

Spontaneous Networking

- Integração dinâmica de dispositivos fixos e móveis em redes ad hoc.
- Exemplo: comunicação entre smartphones e IoT sem configuração prévia.

Módulo 4 – Modelo Cliente/Servidor

Introdução

- O **modelo cliente/servidor** é composto por:

- **Servidores:** processos que oferecem serviços.
 - **Clientes:** processos que requisitam esses serviços.
- Baseia-se em um **protocolo de comunicação requisição/resposta**, que organiza a interação de forma simples e direta.

Protocolo Requisição/Resposta

- **Estrutura:**
 - Cliente envia uma **requisição**.
 - Servidor processa e envia uma **resposta**.
- **Características:**
 - Protocolo **sem conexão** (cada interação é independente).
 - Tipicamente implementado em **três camadas**: física, enlace e requisição/resposta.
- **Comparação com o modelo OSI (ISO):**
 - Dispensa camadas 3 (rede) e 4 (transporte), mas elas são desejáveis para **roteamento** e **confiabilidade**.
 - **Vantagem:** maior **eficiência** → menos camadas = menor overhead.

Comunicação

- Implementada com apenas dois serviços:
 - **send(dest, &buffer)** → envio de mensagem.
 - **receive(dest, &buffer)** → recepção de mensagem.
- Esses serviços abstraem os detalhes da rede, dando uniformidade ao modelo.

Registro e Lookup

- **Registro do servidor:** antes de operar, o servidor deve se registrar no sistema, informando seus dados de identificação.
- **Lookup do cliente:** só é possível usar serviços de servidores **ativos e registrados**. → Garante que clientes não tentem se comunicar com servidores inexistentes.

Estrutura de Processos

- **Servidor (monolítico no exemplo):**
 - Loop infinito de espera → recebe requisição → processa → envia resposta.
 - Pode ser estendido para arquiteturas **multithreaded** ou distribuídas.
- **Cliente:**
 - Monta a requisição (com identificador e tipo de serviço).
 - Envia ao servidor.

- Aguarda a resposta.

Problemas Fundamentais

Localização do Servidor (Endereçamento)

- **Questão:** como o cliente descobre em qual máquina está o servidor?
 - Solução: uso de **servidores de nomes**, diretórios distribuídos ou mecanismos de **binding** dinâmico.
1. Endereçar máquinas: associar estaticamente as máquinas aos servidores e colocar na mensagem o endereço da máquina
 - Consiste em se definir de maneira estática em que máquinas estão quais servidores.
 - Ao entrar no ar, o cliente tem acesso a esta configuração estática.
 - Quando deseja solicitar um serviço, o cliente coloca no cabeçalho da mensagem o endereço numérico que é o endereço físico da máquina onde o processo servidor se executa
 - Desvantagens:
 - O sistema operacional deve abrir a mensagem para identificar para qual servidor é endereçada a mensagem que chegou.
 - Aumento do overhead de tratamento da mensagem
 2. Endereçar processos: colocar na mensagem a identificação do processo servido
- Endereçamento por nomes únicos
 - A atribuição de nomes numéricos únicos necessita de um processo para decidir que nome deve ser atribuído a que processo.
 - Como fazer o mapeamento processo x nome?
 - processo alocador de endereços centralizado
 - escolha randômica de nomes
 - servidor de nomes
 - Auxílio de hardware específico

Escolha randômica de nomes

- Cada processo obtém seu identificador único (nome numérico) randômicamente a partir de um espaço de endereçamento grande (ex: 64 bits).
- Problema: Em que máquina está o processo?

Servidor de nomes

- A cada serviço é atribuído um nome ascii. Um processo chamado servidor de nomes é o responsável pela manutenção da relação nome do serviço x endereço
- O servidor de nomes pode ser centralizado ou distribuído

Auxílio de hardware de rede

- Cada chip de interface de rede mantém a relação endereço do processo x endereço da máquina. O próprio chip verifica se o processo se encontra na sua máquina

Abordagem mais utilizada

Servidor de nomes (centralizado ou por subconjuntos de máquinas) e caches locais

Estrutura do Servidor

- **Monolítico**: único processo que trata todas as requisições.
- **Multithreaded**: cada requisição pode ser atendida por um thread, aumentando concorrência e escalabilidade.

Servidores multithreaded

- Os servidores multithreaded surgiram para agilizar o tratamento de requisições.
- Neste modelo, diversas requisições podem estar sendo tratadas ao mesmo tempo.
- Os servidores multi-processos também permitem o tratamento simultâneo das requisições, porém a um custo de gerenciamento mais alto
- Modelos com threads fixas
 - Dispatcher/worker
 - Team
 - Pipeline
- Modelos com threads variáveis
 - Thread per request
 - Thread per connection
 - Thread per object
- A thread dispatcher (I/O thread) cria uma nova thread trabalhadora para cada requisição
- A trabalhadora se auto-destrói após o término do processamento da requisição

Thread per request

- A thread dispatcher (I/O thread) cria uma nova thread trabalhadora para cada requisição
- A trabalhadora se auto-destrói após o término do processamento da requisição
- A thread dispatcher (I/O thread) cria uma nova thread trabalhadora para cada requisição
- A trabalhadora se auto-destrói após o término do processamento da requisição

Thread per connection

- Neste caso, a cada conexão é associada um thread.
- O servidor cria um thread quando o cliente estabelece uma conexão e a destrói quando a conexão é finalizada.
- Todas as solicitações para uma mesma conexão são enfileiradas
- Em relação à abordagem thread per request, esta abordagem reduz o overhead de gerenciamento de threads.
- No entanto, as requisições de alguns clientes podem ficar enfileiradas quando existem threads ociosas no servidor.

Thread per object

- Neste caso, um thread é associada a cada objeto envolvido no tratamento da requisição.
- A thread de I/O recebe as solicitações e as encaminha aos threads associados aos objetos.
- Este modelo visa reduzir a contensão no acesso a mecanismos de sincronização.
- Da mesma maneira que a abordagem thread per connection, esta abordagem reduz o overhead de gerenciamento de threads, em relação à abordagem thread per request.
- No entanto, da mesma forma, as requisições de alguns clientes podem ficar enfileiradas quando existem threads ociosas no servidor

Semântica da Comunicação

A comunicação por troca de mensagem é composta por duas partes: uma que quer enviar mensagens e outra que deseja recebê-las. A comunicação somente ocorre quando as duas partes estão prontas.

- **Blocante (síncrona):** cliente/servidor aguardam até que a operação seja concluída.
- **Não-blocante (assíncrona):** envio/recepção retornam imediatamente, aumentando paralelismo, mas exigindo mecanismos de controle. → Essa escolha impacta diretamente em **desempenho e robustez**.

Primitivas Bloqueadas

- A comunicação que utiliza primitivas bloqueadas é dita comunicação síncrona porque, no momento da troca de mensagens, cada processo envolvido na comunicação sabe exatamente em que estado da execução se encontra o outro processo (ele está executando a primitiva de comunicação).
- Assim, além de comunicar dados, os processos trocam implicitamente informações sobre o seu estado de execução
- A comunicação síncrona combinada aos threads de comunicação chama-se comunicação pseudo-assíncrona.
- Este tipo de comunicação está sendo bastante estudado pois engloba vantagens tanto da comunicação síncrona como da comunicação assíncrona.

Primitivas não-bloqueadas

- No caso do send antes do receive, onde guardar a mensagem até que o receive correspondente seja executado?

- manter a mensagem no próprio espaço de endereçamento do processo
- copiar a mensagem para uma área interna do sistema operacional

Manter a mensagem na área do processo

O processo não pode reutilizar o buffer de mensagem até ter certeza de que a mensagem foi transmitida ao processo receptor.

Manter a mensagem na área do processo

- Como saber se a mensagem já foi enviada? | Interrupção do processo emissor no momento da recepção da mensagem
- Primitiva adicional de teste de envio de mensagem:
 - `i = send_assinc(msg);`
 - `test_send(i);`

Manter a mensagem na área do processo

- Como saber se a mensagem já foi enviada?
- Interrupção do processo emissor no momento da recepção da mensagem
- Primitiva adicional de teste de envio de mensagem:
 - `i = send_assinc(msg);`
 - `test_send(i);`

Primitivas Bloqueadas

- + Semântica mais simples
- Possibilidade de deadlock
- Perda de concorrência

Primitivas não-bloqueadas

- + Alto potencial de concorrência
- Semântica complexa

A maioria dos sistemas permite tanto a comunicação síncrona como a comunicação assíncrona. O programador escolhe entre as duas ou pode usar uma combinação delas, por exemplo, `send` não-bloqueado e `receive` bloqueado.

Síntese

O modelo **Cliente/Servidor** é a base dos sistemas distribuídos modernos, pela simplicidade e clareza do protocolo requisição/resposta.

- Seus desafios principais são:
 - **Localização do servidor** (endereçamento).
 - **Estrutura interna** (monolítico vs multithreaded).
 - **Semântica da comunicação** (bloqueante vs não-bloqueante).
- É essencial para compreender tecnologias como **NFS**, **RPC** e **serviços de diretório**, que se apoiam nesse paradigma.

Primitivas Bufferizadas x Primitivas não Bufferizadas

Questão: Onde armazenar as mensagens que chegam no servidor enquanto este está tratando outra mensagem?

Soluções:

1. A mensagem é descartada.
2. O kernel da máquina receptora guarda a mensagem por um tempo.
3. Envio de mensagens a estruturas de dados e não a processos.
4. Bloquear o envio quando não há espaço no destino.

Solução 1 – A mensagem é descartada

- kernel da máquina receptora verifica se o servidor está esperando a mensagem.
 - Se sim, entrega a mensagem ao servidor.
 - Caso contrário, descarta a mensagem.
- cliente retransmite-a por timeout.
- Problemas: sobrecarga da rede, postergação indefinida, detecção de saída de servidor por timeout.

Solução 2 – A mensagem é guardada por um tempo

- kernel da máquina receptora guarda a mensagem por um tempo. Se, após este tempo, a mensagem não tiver sido “recebida”, ela é descartada.
- Problema: necessidade de gerenciamento de buffers.

Solução 3 – Envio de mensagens a estruturas de dados

- Neste caso, a mensagem é enviada a uma estrutura de dados gerenciada pelo kernel e não diretamente ao processo servidor.
- Problemas: Gerenciamento de buffers, tratamento de buffer cheio.

Solução 4 – Bloquear o envio para buffer cheio

- Neste caso, o envio de mensagens, apesar de não bloqueado, pode ser blocante no caso de buffer cheio.
- Isso torna mais complexa a semântica da primitiva.

Processos envolvidos na comunicação

- 1 a 1: O processo x envia uma mensagem ao processo y. O processo y recebe somente a mensagem do processo x.
 - Exemplo: `send(y, &msg); receive(x, &msg);`

- n a 1: O processo x envia uma mensagem ao processo y. O processo y recebe mensagem de qualquer processo.
 - Exemplo: `send(y, &msg); receive(ANY, &msg);`
 - Mecanismos: portas (SO), caixas-postais restritas.
- n a n: O processo x envia uma mensagem a qualquer processo. O processo y recebe mensagem de qualquer processo.
 - Exemplo: `estrutura_y = aloca_caixa_postal(); send(estrutura_y, &msg); receive(estrutura_y, &msg);`
 - Mecanismos: caixas-postais genéricas.

Confiabilidade da Comunicação

- Problemas que devem ser tratados quanto à confiabilidade: perda de mensagens, inversão na ordem das mensagens, mensagens corrompidas.

Perda de mensagens

- Problema: Uma mensagem enviada de x para y pode ser perdida no meio do caminho.

Perda de mensagens – Acknowledgements (ACKS)

- Kernel da máquina receptora envia um ACK à máquina transmissora cada vez que uma mensagem é recebida.
- que confirmar (ACK)?
 - Pacotes: Ao ser enviada, uma mensagem é geralmente quebrada em n pacotes de tamanho fixo. A cada pacote recebido, é enviado um ACK (alto overhead e retransmissão de pacotes).
 - Mensagens inteiras: Overhead mais baixo e retransmissão de mensagens.

Inversão na ordem das mensagens

- Problema: Duas mensagens enviadas de x para y podem ser recebidas em uma ordem diferente da de envio.
- Solução: Inclusão de numeração lógica (timestamps lógicos) no header da mensagem.

Mensagens Corrompidas

- Problema: Uma mensagem enviada de x para y com um conteúdo c' pode ser recebida em y com o conteúdo c''.

Mensagens corrompidas

- Fonte de corrupção da mensagem: meio físico de transmissão, detecção por algoritmos padrão (CRC, bit de paridade, etc.) e agente externo.
- A mensagem pode ter sido deliberadamente alterada (falha maliciosa).

- A detecção de falhas maliciosas é bem mais complicada.

Módulo 5 – Network File System (NFS)

Introdução

Proposto inicialmente pela Sun Microsystems para sistemas Unix, no início da década de 1990.

Suporte a sistemas heterogêneos (software e hardware).

Utiliza a RPC para comunicação entre cliente e servidor.

Características básicas

Objetivo: Compartilhamento de Arquivos.

Arquitetura: Cliente/Servidor.

Servidor NFS: exporta diretórios para disponibilizá-los a clientes remotos.

Lista de diretórios exportados: /etc/exports.

Comando: exportfs.

Cliente NFS: monta os diretórios disponibilizados no servidor.

Os arquivos montados com sucesso são automaticamente compartilhados.

Protocolos do NFS

Protocolo: conjunto de mensagens enviadas entre o cliente e o servidor. Para cada mensagem, o servidor executa um conjunto de rotinas.

O NFS possui dois protocolos: montagem de arquivos e acesso a arquivos e diretórios.

NFS – Montagem de Diretórios

- 1) O cliente envia um path ao servidor e pede permissão para montá-lo.
- 2) O servidor verifica se o path existe e se ele foi exportado.
- 3) O servidor envia a autorização para manipulação de arquivo.

Autorização para manipulação de arquivos: contém tipo de sistema de arquivo, disco, número do i-nodo, segurança etc.

Todos os acessos ao arquivo serão via os dados contidos na autorização.

NFS – Tipos de Montagem

Montagem manual: feita manualmente sempre que se deseja acessar um diretório remoto.

Montagem automática: feita no momento do boot (/etc/rc).

Automontagem: um conjunto de diretórios remotos é associado a um diretório local. Quando um arquivo remoto for aberto pela primeira vez, o primeiro servidor que responder terá seu diretório montado no cliente.

O usuário deve garantir que os diretórios remotos do conjunto de automontagem serão sempre idênticos (tipicamente READ-ONLY).

NFS – Acesso a Arquivos

As chamadas Unix tradicionais para tratamento de arquivos são aceitas pelo NFS, com exceção de open e close.

Em vez de open, há uma operação de lookup que retorna autorização para manipulação do arquivo (sem gerar entrada na tabela de arquivos).

Read e write: contêm as informações de autorização de manipulação e de leitura/escrita.

O NFS é stateless, ou seja, não depende de estado anterior para execução.

Algumas operações, como bloqueio de arquivos, podem falhar nesse modelo.

NFS – Servidor Stateless

O NFS usa os bits rwx do Unix para proteção e solicita acessos via RPC.

O cliente pode alterar dados na mensagem e burlar o sistema de proteção.

Para resolver, cada cliente possui uma chave criptografada: RPC com criptografia e Kerberos integrado ao NFS.

NFS – Implementação no SunOS

Os arquivos possuem v-nodos. Há um v-nodo por arquivo (local ou remoto).

O v-nodo identifica um arquivo virtual, que pode apontar para um i-nodo (local) ou r-nodo (remoto).

O r-nodo contém as informações da autorização de manipulação do arquivo.

Caching

Para melhorar o desempenho, são usadas buffer caches em memória RAM.

Read-ahead: antecipa leituras buscando dados próximos aos últimos acessados.

Delayed-write: escreve dados alterados em memória e depois em disco.

Operação sync: grava em disco os dados modificados há mais de 30 segundos (padrão).

Caching no Servidor NFS

A buffer cache é usada na leitura de forma similar ao Unix.

Na escrita, há duas opções:

Write-through: dados são escritos na cache e no disco.

Write-back: dados ficam na cache e vão ao disco somente após commit.

Caching no Cliente NFS

Os resultados de read, write, getattr, lookup e readdir são mantidos na cache do cliente.

Para garantir consistência, o cliente pode consultar o servidor (polling).

Polling do Servidor - Leituras

Cada dado na buffer cache possui dois timestamps:

Tc: tempo da última validação; Tm: tempo da última modificação no servidor.

O dado está válido se $T - T_c < t$ OU $T_{mcliente} = T_{mservidor}$.

$T_{mservidor}$ é checado a cada t (3s a 30s, no Solaris).

Se $T_{mservidor} \neq T_{mcliente}$, o dado é buscado novamente.

Polling do Servidor - Escritas

Quando o dado é modificado na buffer cache, ele é marcado como dirty.

Os dados são enviados ao servidor quando ocorre sync ou fechamento do arquivo.

NFS – Resumo de Características

Transparência de acesso: Sim.

Transparência de localização: pode ser implementada, mas não é obrigatória.

Transparência de migração: Não.

Escalabilidade: Limitada.

Replicação de arquivos: R/O files, R/W com NIS.

Heterogeneidade de hardware: Sim.

Heterogeneidade de sistema operacional: Sim.

Tolerância a falhas: Sim (stateless execution).

Consistência: Limitada.

Segurança: Sim, com Kerberos e RPC-SEC.

Eficiência: Sim.

Módulo 6 – Ordenamento de Eventos

Notações Básicas

- Um **sistema distribuído** é composto por **N processos** P_i , cada um executado em um processador distinto.
- **Não há memória compartilhada** → a comunicação ocorre **exclusivamente por envio e recepção de mensagens**.
- Cada processo possui um **estado interno (si)**, que inclui valores de suas variáveis e é modificado durante a execução.
- As ações possíveis são:
 - **send**: envio de mensagem.
 - **receive**: recepção de mensagem.
 - **operações internas**: modificações do estado local.
- Cada ação é tratada como um **evento**.

Sequência de Eventos

- A execução de um processo pode ser representada por uma relação totalmente ordenada ($\rightarrow_i$) entre eventos.
- Exemplo: se em P_i temos eventos e e e' , então $e \rightarrow_i e'$ se e ocorre antes de e' .
- O **histórico de um processo (H_{P_i})** é a sequência de seus eventos ordenados.

Exemplo no material:

- H_{P1} : $a \rightarrow b \rightarrow c \rightarrow d$
- H_{P2} : $e \rightarrow f \rightarrow g$
- H_{P3} : $h \rightarrow i \rightarrow k \rightarrow l \rightarrow m \rightarrow n$

Ordem “Happens-Before” ($\rightarrow$)

Lamport introduziu a relação “**happens-before**” para ordenar eventos **em diferentes processos**.

Um evento a precede b ($a \rightarrow b$) se:

1. a e b pertencem ao mesmo processo e a ocorre antes de b .
2. a é o envio de uma mensagem e b é a recepção da mesma mensagem.
3. Existe transitividade: se $a \rightarrow c$ e $c \rightarrow b$, então $a \rightarrow b$.

Essa ordem é também chamada de **ordem causal**, pois reflete a dependência de causa e efeito.

Concorrência de Eventos

- Se dois eventos não têm relação definida pela ordem **happens-before**, eles são ditos **concorrentes**: $a \parallel b$.
- **Eventos concorrentes** não possuem uma ordem causal → podem acontecer em qualquer ordem.
- Isso reflete a natureza dos sistemas distribuídos: a ausência de um **clock global** impede ordenação absoluta.

Síntese

- Cada processo tem sua linha temporal (sequência de eventos).
- A relação **happens-before** conecta eventos entre processos, permitindo comparar quando há dependência de mensagens ou execução.
- Eventos que não podem ser relacionados são **concorrentes**, e o sistema não impõe ordem entre eles.

→ Essa noção é fundamental para algoritmos distribuídos como **relógios lógicos de Lamport** e **relógios vetoriais**, usados para garantir **consistência causal** em sistemas distribuídos.

Modulo 7 - Remote Procedure Call (RPC)

Introdução

- A troca de mensagens é o paradigma de programação que mais se aproxima da arquitetura genérica de sistemas distribuídos.
- Em sistemas com máquina única, no entanto, raramente nós utilizamos este paradigma.
- Muitos programadores acham que a programação por troca de mensagens é difícil.
- No intuito de tornar a programação de sistemas distribuídos mais próxima da programação dos sistemas monoprocessadores, foi proposta a chamada a procedimento remoto (Remote Procedure Call) - RPC.

Introdução

- A RPC permite que processos executem funções pertencentes a processos que rodam em outra máquina.

Introdução – Chamada a Procedimento Local

- Chamada local a procedimento:
- Em C, os parâmetros podem ser passados por duas maneiras:
- por valor: Exemplo: `calcula_raiz(x);`
- por referência: Exemplo: `calcula_raiz(&x);`

Introdução- Chamada a Procedimento Local

- (Conteúdo referente à pilha de execução antes, durante e após a chamada — mantido apenas o texto descritivo implícito nos slides anteriores; detalhes visuais de pilha foram omitidos conforme regra de remoção de diagramas.)

Funcionamento da RPC

- A RPC deve ser, o máximo possível, transparente ao programador.

- O programador não deve notar que está chamando uma rotina que se encontra em outra máquina.
- A chamada ao procedimento remoto é idêntica a uma chamada local.

LPC vs RPC

- (Conteúdo comparativo textual já coberto nos slides anteriores; não há texto adicional além da ilustração, que foi removida conforme regra.)

Funcionamento da RPC

- Como fazer chamadas a processos que estão em máquinas remotas?
- Devemos introduzir procedimentos adicionais (stub) para serem chamados pelos processos cliente e servidor.
- Os procedimentos stub transformam a chamada a procedimento em troca de mensagens, de maneira transparente.

Funcionamento da RPC

- 1. O cliente faz chamada a um procedimento remoto
- 2. É executado um procedimento (stub de cliente) que empacota os parâmetros e chama o kernel para enviar uma mensagem ao servidor.
- 3. O kernel origem envia a mensagem ao kernel destino.
- 4. O kernel destino chama o stub do servidor, passando a mensagem.
- 5. O stub do servidor recebe a mensagem, desempacota-a, e chama o procedimento remoto no servidor.

Funcionamento da RPC

- 6. O servidor executa a solicitação e devolve o resultado ao stub do servidor.
- 7. O stub do servidor empacota a mensagem e a envia ao kernel.
- 8. O kernel do servidor envia a mensagem ao kernel do cliente.
- 9. O kernel do cliente passa a mensagem ao stub do cliente.
- 10. O stub do cliente desempacota os resultados e os passa ao cliente.

Funcionamento da RPC

- Stub cliente:
- Recebe os parâmetros, empacota-os e envia a mensagem ao stub servidor
- Ordenação de parâmetros: ordem de empacotamento de parâmetros
- Na mensagem, são colocados os parâmetros e o nome do procedimento remoto a ser executado
- O stub cliente faz parte do código do cliente!!!

Funcionamento da RPC

- Stub servidor:
- Recebe a mensagem do kernel e faz a chamada ao procedimento a ser executado no servidor.
- Recebe o resultado do servidor e o empacota para que o kernel o envie ao cliente.
- O stub servidor faz parte do código do servidor!!!

RPC – Interpretação de Parâmetros

- Problema: Como garantir que o dado enviado para uma máquina será interpretado da mesma maneira na máquina destino?
- Em redes homogêneas, não há problema.

- Todos os tipos básicos (int, float, char) são armazenados da mesma forma.
- Como proceder no caso de redes heterogêneas?

RPC – Interpretação de Parâmetros

- Representação interna dos tipos básicos:
- Mainframe IBM: EBCDIC x IBM-PC: ASCII
- Representação de inteiros: complemento a 1 x complemento a 2
- Representação de ponto flutuante (mantissa e expoente): formato IEEE e formatos proprietários.
- Ordenação de bytes: esquerda para a direita (big endian): SPARC x direita para a esquerda (little endian): Intel 386.

RPC – Interpretação de Parâmetros

- Solução 1:
- receber os parâmetros, empacotá-los de acordo com uma representação padrão, enviá-los. Na recepção, converter da representação padrão para o formato da máquina local
- Ineficiente se máquinas forem do mesmo tipo.
- Solução 2:
- enviar a msg no formato nativo, mas com um campo informando qual o formato utilizado.
- Perda da flexibilidade.

RPC – Interpretação de Parâmetros

- A solução 1 é de longe a mais adotada.
- Marshalling: transformar uma coleção de dados do formato nativo para o formato padrão
- Unmarshalling: transformar uma coleção de dados do formato padrão para o formato nativo
- External Data Representation (XDR): padrão aceito entre as partes para a representação de estruturas de dados e valores primitivos

LPC vs RPC

- (Conteúdo já abordado anteriormente; slide ilustrativo removido conforme regra.)

RPC- Passagem de parâmetros

- Como passar parâmetros por referência (ponteiros)? Exemplo: rotina calculo(&a,&b)
- Proibir a passagem de ponteiros
- O stub cliente acessa o endereço &a e copia o valor de a para a mensagem. O stub servidor recebe a mensagem e chama a rotina do servidor, utilizando agora um endereço interno ao stub (onde o parâmetro foi colocado, dentro da mensagem recebida do cliente). Isso funciona no caso de inteiros e no caso de strings dos quais sabemos o tamanho. Não funciona para ponteiros genéricos.

RPC- Geração de Stubs

- Os programadores compõem um arquivo com o procedimento remoto a ser executado no servidor e um gerador de código automático (compilador) gera programas C que contêm os stubs. No NFS, o utilitário se chama RPCGEN.

RPC – Geração do Programa Cliente e do Programa Servidor

- (Slide descritivo de composição de arquivos; mantido apenas o entendimento implícito nos slides anteriores.)

RPCGEN- Exemplo

- Exemplo de especificação de rotina em linguagem RPC, que será utilizada pelo RPCGEN para gerar os stubs cliente e servidor:

```
program BINOP {  
  version BINOP_VERS {  
    long BINOP_ADD (struct input_args) = 1;  
  } = 300030;  
}  
struct input_args {  
  long a; long b;  
};
```

RPC- Ligação dinâmica

- Como um cliente localiza o servidor? A maneira mais simples é associar estaticamente o endereço do servidor ao seu nome, gravando no hardware do cliente o endereço de rede do servidor. Esta abordagem é bastante inflexível.
- Com a ligação dinâmica a correspondência entre o nome do servidor e seu endereço é feita dinamicamente.

RPC- Ligação dinâmica

- Rodar o servidor. Antes de entrar no loop principal, o servidor chama a rotina initialize.
- rotina initialize: exporta a interface do servidor
- O servidor envia uma mensagem contendo o seu nome, a versão, um identificador único e o seu endereço para o binder. Este processo é chamado de registro do servidor.

RPC- Ligação dinâmica

- Rodar o cliente. Ao executar o primeiro procedimento remoto, o stub do cliente verifica que não há nenhum servidor ligado a ele.
- Envia, então, uma mensagem ao binder, pedindo a importação da interface do servidor.
- O binder verifica se há algum servidor “exportado”. Se sim, ele fornece ao stub do cliente, o identificador e o endereço do servidor.

RPC- Ligação dinâmica

- Vantagens:
 - flexibilidade
 - tolerância a falhas
- Desvantagens:
 - overhead
 - o ligador, se centralizado, torna-se facilmente um gargalo. Ligadores distribuídos tem o problema de coerência.

RPC- Produtos comerciais

- ONC RPC - RPC da Sun (padrão)
- Primeiro sucesso comercial

- O NFS foi implementado com a ONC RPC
- TI RPC (transport-independent RPC): RPC independente do protocolo de transporte. O TI RPC pode usar tanto o TCP/IP (confiável) como o UDP/IP (não-confiável)
- DCE RPC - Evolução da ONC RPC

RPC- Tratamento de falhas

- O principal problema do modelo RPC é a manutenção da transparência na presença de falhas.
- Tipos comuns de falhas:
 - O cliente não localiza o servidor
 - Perda de mensagens solicitando serviços
 - Perda de mensagens com resposta
 - Queda do servidor
 - Queda do cliente

RPC- Tratamento de falhas O cliente não acha o servidor

- Essa situação pode ocorrer se o servidor não está no ar ou a versão do cliente é diferente da versão do servidor. O procedimento remoto deve retornar um erro.

RPC- Tratamento de falhas O cliente não acha o servidor

- Que erro retornar?
- Como no Unix, pode retornar -1 e setar a variável global errno com o erro correspondente à "servidor inválido".
- Problema: -1 pode ser uma resposta válida para alguma função.
- Gerar uma interrupção, inventando um novo sinal SIGNOSERVER.
- Problema: várias linguagens não suportam sinais

RPC- Tratamento de falhas O cliente não acha o servidor

- Nas duas soluções anteriores há uma perda da transparência. O modelo de RPC diz que a chamada remota deve ser exatamente igual à chamada local. Em uma chamada local, o erro NOSERVER nunca seria retornado!!!

RPC- Tratamento de falhas Perda da msg de solicitação

- O cliente manda executar um procedimento remoto, mas esta solicitação se perdeu na rede. Neste caso, o cliente pode ficar esperando indefinidamente.

RPC- Tratamento de falhas Perda da msg de solicitação

- Solução:
 - Uso de temporizador no cliente. O temporizador é setado a cada chamada. Se o tempo se esgotar, é feita a retransmissão. Se várias retransmissões são feitas em um curto espaço de tempo, o cliente conclui que o servidor caiu.
- Problemas:
 - overhead de processamento e de mensagens, caso o timeout seja mal estipulado

RPC- Tratamento de falhas Perda da resposta

- O servidor executou a função remota e enviou a resposta. Só que a resposta não chegou ao cliente.

RPC- Tratamento de falhas Perda da resposta

- Retransmitir a solicitação
- Problema: a função será executada duas ou mais vezes, quando na realidade, só deveria ser executada uma vez.
- Retransmitir a solicitação com numeração lógica.
- Permite a distinção entre uma nova solicitação e uma retransmissão.

RPC- Tratamento de falhas Queda do servidor

- O servidor saiu do ar.
- Quando ele saiu do ar?
- Esperando mensagem ou
- Executando o procedimento remoto ou
- Enviando a resposta

RPC- Tratamento de falhas Queda do servidor

- Para decidir o que fazer, deve-se determinar o número de vezes que uma solicitação pode ser executada.
- Executar no mínimo uma vez. O kernel do cliente espera até que o servidor dê um novo boot ou que uma ligação com outro servidor equivalente seja estabelecida. Neste ponto, pede a operação novamente.
- Executar no máximo uma vez. Desistir imediatamente quando der o timeout.
- Executar de 0 a n vezes. Não dar nenhuma garantia para o cliente.

RPC- Tratamento de falhas Queda do cliente

- O cliente enviou uma solicitação e saiu do ar antes que o servidor respondesse.
- Neste caso, nós temos um procedimento órfão. Um procedimento órfão é executado sem ter ninguém esperando por seu resultado.

RPC- Tratamento de falhas Queda do cliente

- Como tratar os procedimentos órfãos?
- extermínio
- reencarnação
- expiração

RPC- Tratamento de falhas Queda do cliente

- Extermínio: antes de enviar uma mensagem de RPC, o stub cliente grava em um arquivo de log a informação: “enviando RPC”.
- Quando o cliente entrar no ar de novo, todos os órfãos daquele cliente são assassinados (kill).
- Problemas:
- overhead de armazenamento de logs
- os órfãos criados por órfãos (RPC que faz RPC) não serão exterminados

RPC- Tratamento de falhas Queda do cliente

- Reencarnação: Uso de timestamps.
- Quando o cliente se recupera, ele envia um timestamp em broadcast aos servidores (timestamp de início).
- Todas as RPCs em andamento para este cliente são verificadas e as que tiverem sido lançadas em tempo anterior são eliminadas.
- Os resultados contêm também um timestamp.

- Assim, mesmo que alguns órfãos sobrem, suas respostas serão desconsideradas pelo cliente porque elas dizem respeito a uma “outra vida”.

RPC- Tratamento de falhas Queda do cliente

- Expiração: a cada chamada remota é associado um tempo máximo (T) no qual o trabalho deve ser realizado. Se o tempo expirar, os procedimentos são abortados e é preciso solicitá-los novamente. Se o cliente esperar T até entrar no ar, terá certeza de que os órfãos foram mortos.
- Como estipular T?
- Em que pontos da execução é seguro eliminar um processo?

RPC- Implementação Tratamento dos Acks

- O problema a ser tratado aqui é a definição do que confirmar: pacotes ou mensagens?
- Protocolo pare e espere
- Protocolo em rajadas
- Protocolo híbrido

RPC- Tratamento dos Acks Protocolo Pare e Espere

- Um ack é enviado do servidor ao cliente a cada pacote que compõe a mensagem. Só depois do recebimento do ACK é que o próximo pacote é enviado. Se um pacote se perdeu, somente ele é retransmitido. O overhead na rede é grande.

RPC- Tratamento dos Acks Protocolo em rajadas

- Um ack é enviado do servidor ao cliente somente quando todos os pacotes que compõem a mensagem são recebidos. No caso de perda de pacotes, o servidor pode enviar um NAK e neste caso a mensagem toda é retransmitida

RPC- Tratamento dos Acks Protocolo híbrido

- Se não há perda de pacotes, o protocolo híbrido comporta-se exatamente como o protocolo em rajadas.
- No caso de perda de pacotes, o servidor envia um ACK contendo os números dos pacotes que foram recebidos.
- Neste caso, somente os pacotes perdidos são retransmitidos.

RPC- Implementação Saturação do fluxo de controle

- Os chips de interface de rede tem capacidade de armazenamento finita.
- Por isso, não devemos ter um fluxo muito grande de envio de pacotes.
- O erro “excesso de pacotes” é produzido quando um pacote chega e o receptor não está pronto para aceitá-lo. Geralmente, ele está tratando o pacote anterior.
- A RPC deve ser projetada para reduzir a probabilidade de saturação do fluxo de controle

RPC- Implementação Saturação do fluxo de controle

- Protocolo pare e espere: o excesso de pacotes raramente ocorre, pois há um certo sincronismo no envio de mensagens.
- Protocolo em rajadas: pode ocorrer com muita frequência, já que o envio é assíncrono.

RPC- Implementação Saturação do fluxo de controle

- Soluções:
- O transmissor introduz um retardo entre o envio de pacotes (busy waiting ou bloqueio)
- Se o chip tem capacidade de armazenamento de n pacotes, enviar n pacotes de cada vez

RPC- Implementação Caminho crítico

- Para melhorar a performance de um sistema, devemos saber onde mais tempo é consumido. Este lugar pode depender das máquinas nas quais a RPC roda.
- Experimentos na DEC Firefly (5 processadores VAX). Protocolo UDP. Chamadas otimizadas. Busy-waiting.
- Chamada nula: custos maiores para a troca de contexto para o stub servidor e movimento de pacotes para a interface de rede.
- Chamada com 1440 bytes: custos maiores para transmissão pela rede e movimento de pacotes para a interface de rede.

RPC- Implementação Cópias

- Como as operações de cópia (tanto locais como remotas) são operações que envolvem bastante tempo, devemos reduzir o número de cópias no projeto de mecanismos do sistema operacional.
- RPC: pior caso:
- stub cliente → kernel
- kernel → placa da interface de rede
- interface de rede origem → interface de rede destino
- interface de rede → kernel
- kernel → stub servidor

RPC- Implementação Otimização das cópias

- Scatter-gather: implementado no hardware da interface de rede: uma mensagem pode ser montada pela concatenação de buffers. O cabeçalho da mensagem pode estar em um espaço de endereçamento (kernel) e o resto da mensagem em outro espaço de endereçamento (cliente)

RPC- Implementação Otimização das cópias

- Mapeamento do buffer da mensagem na memória do kernel e do cliente. O corpo da mensagem é mantido no espaço de endereçamento do cliente.
- Alinhamento do buffer em relação à página
- Proteção

RPC – Implementação Lightweight RPC

- Caso o cliente e o servidor estejam na mesma máquina, a RPC é implementada exatamente como se fosse remota e somente a troca de mensagens é local.
- Caso fosse testado se cliente e servidor residem na mesma máquina, otimizações poderiam ser aplicadas.
- Uma destas otimizações é a Lightweight RPC.

RPC – Implementação Lightweight RPC

- Na LRPC, se estão na mesma máquina, cliente e servidor comunicam-se por memória compartilhada.

RPC- Níveis de Utilização

- Apesar da filosofia da RPC requerer que a chamada remota se comporte exatamente como uma chamada local, na prática nós podemos utilizar a RPC em 3 níveis:
- Alto nível: chamada transparente
- Nível médio: a rotina é chamada através de callrpc e o servidor se registra
- Baixo nível: controle sobre o protocolo, retransmissões, número de sockets alocados, erros, etc

RPC- Características Adicionais

- Broadcast RPC: O cliente envia um broadcast na rede e vários servidores podem responder.
- Protocolo UDP
- Os servidores só respondem quando tiverem processado a solicitação. Em caso de erro, ficam silenciosos.
- Até quantas respostas esperar?
- Nenhuma, uma, a maioria, todas, etc.

Broadcast RPC

- (Conteúdo ilustrativo removido conforme regra.)

RPC- Características Adicionais

- No-Response RPC ou batching: As RPCs deste tipo assumem que o servidor não deve enviar resposta. Assim, o cliente continua a execução logo depois da chamada à RPC, sem esperar que o seu tratamento seja terminado ou mesmo iniciado.
- Requer o protocolo TCP.
- Aumenta o paralelismo da aplicação distribuída

No Response RPC

- (Conteúdo ilustrativo removido conforme regra.)

Módulo 8 – Distributed Shared Memory

Memória Compartilhada Distribuída

- A memória compartilhada distribuída (Distributed Shared Memory) é uma abstração que cria uma memória global, acessível por todos os processadores.

Memória Compartilhada Distribuída

- A DSM pode ser implementada basicamente por duas abordagens:
- Objetos distribuídos
- Memória virtual compartilhada

DSM- Objetos Distribuídos

- Neste caso, a abstração de memória compartilhada é tratada como um objeto ou um conjunto de objetos.

DSM- Objetos Distribuídos

- As operações definidas sobre o objeto memória são as operações básicas de leitura e escrita. Estas operações serão efetuadas somente no interior do objeto.
- Em geral, as operações sobre um determinado objeto são indivisíveis. Esta suposição simplifica o tratamento da sincronização.

DSM- Objetos Distribuídos

- Como implementar os objetos distribuídos?
- Objetos centralizados: todas as referências remotas são via mecanismos de comunicação (e.g. RPC)
- Objetos migratórios: uma referência remota faz com que o objeto migre para o nodo que o referenciou
- Objetos replicados: cada referência remota cria uma cópia do objeto no nodo local

DSM- Objetos Distribuídos Exemplos

- Orca
- Linguagem de programação orientada a objetos desenvolvida pelo grupo de Bal e Kaashoek na Holanda
- Aurora
- Desenvolvida na Universidade de Toronto, propõe o comportamento dinâmico

DSM- Memória Virtual Compartilhada

- Proposta por K. Li em Princeton, 1986
- A recuperação de dados é iniciada por page faults e feita por paginação entre as memórias locais

DSM- Memória Virtual Compartilhada

- A programação por SVM pode ser feita de maneira completamente transparente
- As operações de acesso à abstração de memória compartilhada são as operações de LOAD e STORE.

Memória Virtual Compartilhada Localização da página

- Quando uma página não local é referenciada, ocorre um page fault. Em uma única máquina, se a página não está em memória local, ela está em disco.
- Em um ambiente DSM, a página pode estar em qualquer uma das memórias remotas ou no sistema de E/S.

DSM- Memória Virtual Compartilhada

- Localização da página
- Servidor centralizado
- Servidor distribuído fixo
- Servidor distribuído dinâmico

Memória Virtual Compartilhada Localização da página

- Servidor centralizado

Memória Virtual Compartilhada Localização da página

- Servidor distribuído fixo

- Os processadores são numerados de 0 a n-1
- Cada processador é o responsável pelo gerenciamento das páginas onde $(\text{page \% n}) = \text{número do processador}$.

Memória Virtual Compartilhada Localização da página

- Servidor distribuído fixo
- Supondo um sistema com 10 processadores:
- O processador 0 é o responsável pela gerência das páginas 0, 10, 20, 30
- O processador 1 é o responsável pela gerência das páginas 1, 11, 21, 31
- etc

Memória Virtual Compartilhada Localização da página

- Servidor distribuído dinâmico
- Só participam da gerência da página os processadores que já a tiveram ou a possuem em suas memórias.

Memória Virtual Compartilhada Localização da página

- Servidor distribuído dinâmico
- Na realidade, nós temos uma lista de prováveis proprietários da página (probable owner).
- Se um nodo contido nesta lista não possuir a página, é garantido que o forward que ele vai oferecer leva ao proprietário real
- Podem ser necessários vários forwards até que se chegue a um dono real da página

Memória Virtual Compartilhada Localização da página

- Considerações
- O servidor centralizado tem os problemas clássicos de gargalo e tolerância a falhas
- O servidor distribuído dinâmico guarda dependência residual
- O servidor distribuído fixo apresenta o melhor compromisso entre o desempenho e a simplicidade de implementação

Memória Distribuída Compartilhada- Consistência

- Tanto a abordagem por objetos distribuídos como a abordagem SVM sofrem de problemas de desempenho relacionados à consistência dos acessos aos dados compartilhados.
- Consistência da memória: ordem na qual o conjunto de acessos à memória são percebidos pelo programador.

Memória Distribuída Compartilhada- Consistência

- Coerência de cache: todas as cópias de um mesmo dado devem possuir o mesmo valor (quando observadas)
- Consistência da memória: diz respeito à ordem na qual o conjunto de acessos à memória são percebidos pelo programador. O problema de consistência da memória engloba o problema de coerência de cache

Memória Distribuída Compartilhada- Consistência

- Modelo de consistência de uma memória única: as operações de acesso à memória são percebidas pelo programador como feitas atomicamente, na ordem especificada pelo programa (consistência sequencial)

- Mantendo esse modelo, várias otimizações foram feitas nos monoprocessadores: write buffers e re-escalonamento de instruções

Memória Distribuída Compartilhada- Consistência

- Para manter a consistência sequencial em uma máquina paralela, essas otimizações internas do processador não podem ser utilizadas.
- Todos os acessos à memória virtual compartilhada devem ser feitos na ordem do programa em todas as memórias locais → graves problemas de desempenho

Memória Distribuída Compartilhada- Consistência

- Os primeiros sistemas implementam a consistência forte (Ivy, Mirage). Baixo desempenho
- Proliferação de modelos de consistência menos fortes
- consistência sequencial
- consistência causal
- consistência do processador
- memória lenta
- consistência do release
- consistência de entrada

Modelos de Consistência-Formalização

- Um programa paralelo é executado por um conjunto de processadores $P = \{p_1, p_2 \dots p_x\}$ que acessam uma memória global compartilhada (M).
- Cada processador p_i possui sua memória local m_i , que é uma cache de todas as posições de M.

Modelos de Consistência-Formalização

- Cada operação é iniciada e depois terminada.
- Existem duas operações não atômicas básicas em M: leitura (r) e escrita (w).
- Uma escrita $w(x)1$ é na verdade um conjunto de escritas do valor 1 na posição x de cada memória m_i .

Modelos de Consistência Consistência Atômica

- A consistência atômica é o modelo mais forte e mais antigo
- Todos os processadores percebem todos os acessos na mesma ordem (ordem global)
- Leva em consideração o tempo físico para dividir a execução de operações em intervalos

Modelos de Consistência Consistência Sequencial

- A Consistência sequencial foi definida por Lamport: um sistema é consistente segundo a consistência sequencial se o resultado de uma execução é equivalente à execução de todas as operações segundo uma ordem sequencial e as operações de cada processador aparecem nesta ordem sequencial segundo a ordem do seu programa.
- Ordem do programa: se o_1 e o_2 são efetuadas pelo mesmo processador p_i e o_1 vem antes de o_2 no código de p_i , então o_1 precede o_2 segundo a ordem do programa

Modelos de Consistência Consistência Sequencial

- Na consistência sequencial, todos os processadores devem perceber a mesma ordem de todos os acessos à memória (ordem global)

Modelos de Consistência Consistência Causal

- A Consistência causal é um modelo de consistência relaxado. Não é mais necessário que os processos percebam a mesma ordem de todos os acessos (ordem parcial)
- A Consistência causal segue a relação de causalidade potencial definida por Lamport. Nesta relação, se um valor escrito por um processador ($w(x)v$) é lido por outro processador ($r(x)v$) então $w(x)v$ precede $r(x)v$ segundo a ordem causal.

Modelos de Consistência Consistência Causal

Modelos de Consistência Consistência do Processador

- A Consistência do Processador é na realidade uma família de modelos de Consistência (Pipelined RAM, PCGoodman, etc).
- Em PRAM, todos os processadores devem ver todos os writes feitos por um mesmo processador na ordem do seu programa.

Modelos de Consistência Consistência PRAM

Modelos de Consistência Consistência PCGoodman

- A Consistência PCGoodman é a consistência PRAM acrescida da coerência de cache.
- PCGoodman é então mais forte que a consistência PRAM.
- Outros modelos da família Processor Consistency: Total Store Order, Partial Store Order.

Modelos de Consistência Memória Lenta

- Este é um dos modelos mais relaxados
- Na memória lenta, só é necessário que os processadores tenham a mesma visão das escritas feitas pelo mesmo processador na mesma posição de memória.
- Não garante a exclusão mútua lógica.

Modelos de Consistência Memória Lenta

Relação entre modelos uniformes

- SC = c. sequencial, PC = c. pcgoodman, CA = c. causal, PRAM = c. PRAM, Lenta = memória lenta

Modelos de Consistência Híbridos

- O paradigma de programação por memória compartilhada possui basicamente duas partes:
- acesso a dados compartilhados
- sincronização
- Existem alguns modelos de consistência, ditos híbridos, que determinam a execução de operações de consistência no momento da sincronização.
- Exemplos de modelos híbridos:

- Weak Ordering
- Release Consistency
- Entry Consistency
- Scope Consistency

Modelos de Consistência Híbridos

- Nos modelos híbridos, nós temos pelo menos 2 ordens:
- ordenação dos acessos básicos em relação aos acessos de sincronização
- ordenação dos acessos de sincronização entre eles

Consistência Fraca

- A consistência fraca, ou weak ordering, divide os tipos de acessos em: leitura (r), escrita (w) e sincronização (sync)
- Quando um sync é executado, todos os acessos anteriores são vistos por todos os processadores e nenhum acesso posterior ao sync é visto por nenhum processador.
- O sync funciona como uma barreira.
- Os acessos de sincronização obedecem a consistência sequencial

Consistência Fraca

Modelos de Consistência Consistência do Release

- A consistência do release é o modelo de consistência mais utilizados atualmente.
- O acesso sync proposto na Consistência fraca é dividido em dois acessos: acquire e release.
- O release garante que os acessos anteriores são terminados.
- As modificações são vistas no momento do release.
- O acquire garante que os acessos posteriores não são começados.

Modelos de Consistência Consistência do Release

- Na consistência do release, os acessos são divididos em:
- ordinários
- leituras
- escritas
- especiais
- não-sincronização
- acquire
- release

Modelos de Consistência Consistência do Release

- Os acessos acquire e release podem ser vistos como a obtenção e a liberação de locks, respectivamente.
- Os acessos especiais de não-sincronização são acessos que se comportam de maneira mais forte que os acessos ordinários, porém não possuem o propósito de sincronização.

Modelos de Consistência Consistência do Release

- A consistência do release necessita que todo acesso à memória compartilhada seja rotulado como leitura, escrita, acquire, release ou nsync.
- Um programa é dito propriamente rotulado se todo acesso compartilhado sem

conflito for rotulado como leitura ou escrita, todo acesso compartilhado onde houver conflito for rotulado como especial e existirem acquies e releases suficientes para delimitar todas as seções críticas

- Foi mostrado que um programa propriamente rotulado na RC comporta-se como se estivesse se executando na consistência sequencial

Modelos de Consistência Consistência do Release

- Condições para a garantia da consistência do release (Rcsc):
- Antes que um acesso ordinário possa ser executado, todos os acessos acquire precedentes devem ter sido terminados
- Antes que um release possa ser executado, todos as leituras e escritas anteriores devem ter sido terminadas
- Acessos especiais obedecem a consistência sequencial.

Modelos de Consistência Consistência do Release

Lazy Release Consistency

- As modificações são visíveis no momento do próximo acquire
- Foi proposta como uma implementação da consistência do release.

Modelos de Consistência Entry Consistency(EC)

- Neste modelo, as atualizações nos dados guardados pelo lock l, só são enviadas para o próximo processo que obtem o mesmo lock.

Modelos de Consistência Scope Consistency(ScC)(1996)

- Neste modelo, as atualizações nos dados que estão na seção crítica limitada pelo lock l, só são enviadas para o próximo processo que obtem o mesmo lock.

Modelos de Consistência Qual escolher?

- A escolha de um modelo de consistência é um compromisso entre a performance e a facilidade de programação
- O desempenho de um modelo de consistência depende de como uma aplicação paralela acessa os dados compartilhados
- Sistemas que suportam múltiplos modelos de consistência
- Midway
- Mermera
- DiVA

Protocolos de Coerência

- O Modelo de Consistência define quando as atualizações devem ser feitas.
- O protocolo de coerência define como as atualizações serão feitas
- Multiple Reader Single Writer (MRSW)
- Multiple Reader Multiple Writers (MRMW)

Protocolos de Coerência MRSW

Protocolos de Coerência MRMW

- W(x)
- twin page
- página com permissão de escrita
- Release

- Codifica as modificações (diffs)
- Broadcast das modificações
- Protege contra a escrita

Memória Compartilhada Distribuída

- Como é a programação da DSM?
- Totalmente transparente (Ivy)
- Com attach e detach de segmentos compartilhados (JIAJIA, TreadMarks)
- Com chamadas à métodos remotos (ORCA)
- Com o auxílio do compilador (Midway)

Programação da DSM Totalmente Transparente

Programação da DSM Attach e detach de segmentos

Programação da DSM Chamada remota a métodos

Programação da DSM Auxílio do Compilador

- Novo tipo de dado

Sincronização em sistemas DSM

- Os mecanismos de sincronização utilizados em sistemas DSM são geralmente os mesmos existentes em sistemas monoprocesados.
- O uso de spin locks em redes de interconexão genéricas pode causar um enorme overhead.
- Locks mutex são geralmente utilizados.

Sincronização em Sistemas DSM

- Principais questões envolvidas:
- Localização dos objetos de sincronização
- Política de atribuição de objetos de sincronização
- Paradigma utilizado para implementar a sincronização

Localização dos Objetos de Sincronização- Locks

- Para localizar os locks, geralmente são utilizados os algoritmos de localização de páginas: servidor centralizado, servidor distribuído fixo ou servidor distribuído dinâmico.
- Como utilizar o algoritmo distribuído fixo, já que ele aplica uma função em um número (número da página) e os locks possuem nomes?
- Associar um número a cada lock
- Aplicar uma função sobre o nome do lock

Localização dos Objetos de Sincronização- Barreiras

- Na maior parte das implementações, as barreiras são gerenciadas por um gerente centralizado.
- A chamada à barreira pode ser:
- Barreira: também conhecida como barreira global de sincronização, espera até que todos os processos da aplicação cheguem à barreira.
- Barreira(n): espera até que n processos da aplicação cheguem à barreira.

Política de atribuição de objetos de sincronização

- O que fazer quando um lock é liberado e existem mais de um processador

esperando?

- Escolher randomicamente o processador que deterá o lock
- Atribuir locks a processadores segundo a ordem FIFO

Paradigma utilizado para implementar a sincronização

- Para a implementação dos objetos de sincronização, podemos escolher entre dois paradigmas:
- Memória compartilhada: usar a própria DSM para implementar a sincronização
- Solução elegante, porém ineficiente
- Troca de mensagens: implementar os objetos de sincronização através de troca de mensagens entre o gerente do lock e o processador da aplicação.
- Solução mais utilizada

Exemplos de Sincronização em DSM

- Ivy: Contador de eventos
- TreadMarks: locks e barreiras
- JIAJIA: locks, variáveis de condição e barreiras
- Brazos: locks e barreiras
- Orca: locks e variáveis de condição
- Midway: locks e barreiras
- Shasta: locks, barreiras e variáveis de condição

Sistemas DSM mais recentes

- Kerrighed (2010): consistência sequencial
- Concordia (2021): consistência forte (strong), uso da placa de rede
- MENPS (2020): release consistency e RDMA